Guion de exposición — Hardware y firmware del entrenador ATmega328P

Equipo: David Orozco, Jose Torres y Leo Rojas.

Duración aproximada: 16–18 minutos, incluida la demostración. El hardware ocupa tres diapositivas breves (unos 2,5 minutos); la mayor parte del recorrido desarrolla el firmware.

Reparto sugerido: David: hardware, arquitectura del programa e inicialización (1–6). Jose: entradas, modos y temporización (7–11). Leo: ISR, salidas, compilación y arranque (12–15). Demostración y conclusiones: equipo (16–17). Anexos: 18–19.

El equipo confirmó que el hardware está terminado. No se inventan mediciones ni resultados nuevos.

1. Entradas, lógica y tiempo real.

David · 35 s. Presentar al equipo y el enfoque: cómo el software convierte las entradas en respuestas y mantiene el refresco de las salidas. El entrenador funciona por sí mismo: el Arduino Uno se usa como herramienta de programación, mientras que el ATmega del entrenador ejecuta el programa. El hardware está terminado, según la confirmación del equipo del 23 de octubre. La portada es una representación conceptual del microcontrolador. Llevar la placa terminada a la exposición.

2. El hardware que controla el programa

David · 40 s. Presentar entradas, microcontrolador y salidas. Las siguientes dos diapositivas explican las decisiones de hardware que permiten ejecutar el programa; después se desarrolla el firmware. El esquema completo queda como anexo para preguntas. El equipo confirmó que el hardware funciona.

3. Energía estable, reloj y reinicio

David · 55 s. Recorrer la fuente en una frase: entrada, diodo, regulador y 5 V. El LM7805 es lineal y disipa energía como calor; 9 V reduce su disipación frente a 12 V. Los valores mostrados son los del diseño, no mediciones nuevas. Diferenciar los electrolíticos de filtrado de los cerámicos de desacoplo. Recordar que el reloj real y la frecuencia de compilación deben coincidir. RESET devuelve el programa a su estado inicial. El esquema incluye además filtro de RESET y auto-reset opcional, que solo se explican si preguntan. No conectar simultáneamente dos fuentes de 5 V.

4. El circuito prepara las señales para el firmware

David · 55 s. Explicar por qué el cursor del potenciómetro proporciona una tensión variable y por qué las pull-up evitan dejar flotantes las entradas abiertas. El firmware invierte la lectura del DIP para representar cerrado como 1 lógico. Los displays son de cátodo común; los transistores conmutan el retorno de cada grupo. Las resistencias limitan la corriente y el programa impide encender los tres grupos simultáneamente en operación normal. El bus usa PD2…PD7 y PB0/PB1; PB2…PB4 seleccionan los grupos. ICSP permite reprogramar sin retirar el micro. No profundizar en fabricación: pasar a cómo el software controla este circuito.

5. Tres responsabilidades bien separadas

David · 55 s. El programa está escrito en C++ para AVR con el entorno Arduino. Combina analogRead y millis con acceso directo a registros. setup se ejecuta una vez y loop se repite. ISR significa rutina de servicio de interrupción: interrumpe brevemente la ejecución normal. No hay hilos ejecutándose simultáneamente; el micro atiende la interrupción y vuelve al punto donde estaba. La división separa el cálculo del contenido de su refresco.

6. setup(): configurar los puertos con máscaras

David · 60 s. Fragmento exacto de setup, sin sus comentarios. 0xFC equivale a 11111100: selecciona seis bits y preserva PD0/PD1. 0x1F selecciona los cinco bits inferiores de B. 0x1E selecciona PC1 a PC4. Una pull-up mantiene una entrada alta cuando el interruptor está abierto. El archivo también activa pull-up en pines sin uso y configura Timer2; no presentar el fragmento como la función completa.

7. De 0…1023 a una lectura estable

Jose · 60 s. El fragmento de leerPot solo adapta el salto de línea para legibilidad. ADC significa convertidor analógico-digital. El promedio reduce variaciones y la histéresis evita actualizar por cambios pequeños; no es una garantía contra cualquier ruido. estable es static: conserva el último valor entre llamadas. La suma máxima es 8184 y cabe en uint16_t. El escalado usa uint32_t para evitar desbordamiento intermedio al multiplicar en AVR. Con pot=512 el display marca 50 y nivel=4 genera 00001111. Las llamadas analogRead son síncronas; no decir que todo el firmware es completamente no bloqueante.

8. El DIP selecciona el modo con dos bits

Jose · 55 s. Línea de retorno de leerDip. Separar nivel eléctrico de significado lógico. La máscara 0x0F elimina los bits ajenos al DIP y 0x03 elige el modo con los dos inferiores. Ejemplo: dip=1010 binario equivale a 10 decimal, pero dip & 3 equivale a 2, por lo que activa la lectura del DIP. No hay rutina explícita de antirrebote digital en este firmware; el filtro descrito antes corresponde al potenciómetro.

9. Cuatro modos, una misma estructura

Jose · 60 s. Explicar el estado de cada modo. El contador usa (cuenta+1)%100. La secuencia cambia sentido en 0 y 7 y luego actualiza la posición. El estado se conserva al cambiar de modo porque no existe una rutina de reinicio por transición; RESET sí reinicia el programa. El modo DIP comparte los dos bits inferiores con la selección del modo, por eso sus valores observables son 02, 06, 10 y 14. No prometer que puede mostrar los 16 valores sin cambiar de modo.

10. Avanzar con millis(), sin esperar con delay()

Jose · 55 s. ultimo es unsigned long y la diferencia unsigned permite comparar tiempos a través del desbordamiento del contador, dentro del intervalo apropiado. No afirmar que el periodo es una temporización de precisión: la lógica comprueba tiempo en cada vuelta, actualiza ultimo al momento actual y no recupera pasos atrasados. Los modos contador y secuencia comparten esta base de tiempo. millis usa la base temporal del núcleo Arduino y el multiplexado usa Timer2. Las conversiones ADC siguen consumiendo tiempo aunque no se use delay.

11. Timer2 genera una interrupción cada 2 ms

Jose · 65 s. Fragmento exacto de setup. CTC significa Clear Timer on Compare Match. WGM21 selecciona el modo; CS22 y CS21 seleccionan división por 256; OCR2A establece el límite; OCIE2A habilita la interrupción de comparación. cli deshabilita interrupciones durante la configuración y sei las habilita después. Las cifras suponen

el reloj de 16 MHz configurado y funcionando. Separar interrupciones por segundo de refrescos de cada grupo. El temporizador marca el instante y la CPU ejecuta la rutina al atenderlo.

12. La ISR actualiza un grupo y regresa

Leo · 65 s. Fragmento exacto de la ISR, sin comentarios. 0x1C selecciona PB2/PB3/PB4. El módulo 3 recorre LEDs, decenas y unidades; con grupo inicial cero, la primera interrupción selecciona decenas, pero el ciclo periódico es el mismo. Apagar antes de escribir evita mostrar el patrón del grupo anterior. La ISR hace poco trabajo: no lee el ADC ni ejecuta la lógica de los modos. volatile indica al compilador que debe efectuar los accesos a memoria porque el contenido se comparte con una interrupción; no vuelve atómica una actualización de todo el búfer. Cada byte se accede individualmente y los tres no se publican como una sola transacción.

13. Del número al patrón de segmentos

Leo · 60 s. Líneas exactas de mostrarNumero. Los bits lógicos corresponden a a,b,c,d,e,f,g,dp. 0x3F enciende a…f para formar cero y deja apagados g y el punto decimal. reordenar recorre los bits activos y usa la tabla de correspondencia. mostrarLeds escribe pantalla[0] con BIT_LED. escribirBus coloca los seis bits inferiores en PORTD y los dos superiores en PORTB; sus máscaras preservan las líneas ajenas al bus. PCB 1 selecciona la correspondencia del diseño; PCB 0 usa orden secuencial. Es una opción de compilación, no una selección durante la ejecución.

14. Del archivo .ino al programa en Flash

Leo · 65 s. Los comandos se ejecutan desde la raíz del repositorio; reemplazar el marcador de puerto por el real, sin los signos de menor y mayor. El perfil Uno selecciona el ATmega328P y 16 MHz; no significa que el entrenador necesite una placa Uno para ejecutar. Compilar traduce y enlaza el programa; programar lo escribe en Flash. El Uno ejecuta ArduinoISP y se comunica mediante ICSP. El software de la aplicación queda en el ATmega, que después funciona con alimentación autónoma. La conexión y alimentación se verifican antes de subir. No ejecutar literalmente el marcador de puerto en una terminal.

15. Firmware, bootloader y fusibles

Leo · 60 s. Explicar los conceptos antes de leer hexadecimales. Los fusibles son bits de configuración, no fusibles físicos. Estos valores son los recomendados en el repositorio, no una lectura nueva del hardware. Grabar un bootloader configura fusibles; hay que comprobar su coherencia si luego se programa por ISP. La carga por ISP puede sobrescribir el bootloader: para volver a cargar por FTDI hace falta un bootloader válido. Los valores del reloj deben coincidir con la frecuencia usada al compilar, porque de ello dependen tiempos y comunicaciones. No cambiar fusibles durante la demostración sin verificar la configuración real.

16. Explicar el código mientras se observa su efecto

Equipo · 2 min. David explica entrada y escalado; Jose contador y refresco; Leo selección DIP, secuencia y RESET. El hardware terminado es la evidencia en vivo. Preparar el contador cerca de 99 y el potenciómetro en velocidad alta para no gastar tiempo esperando. Para el modo DIP dejar SW2 cerrado y SW1 abierto; variar SW4/SW3 como 00,01,10,11. No prometer lectura de 0…15 en ese modo. Mostrar que cambiar la velocidad de la animación no modifica el periodo de Timer2. Las pruebas previas aislaron comunicación, reloj y segmentos antes de integrar la aplicación. No inventar mediciones ni declarar resuelto por una causa específica el antiguo salto observado en video.

17. El firmware coordina datos, estados y tiempo

Equipo · 35 s. Una conclusión por integrante. Destacar cómo cada decisión se ve en una función del programa. El acceso a registros permite controlar bits concretos y la separación entre loop e ISR mantiene el refresco independiente de los modos. Abrir preguntas sobre máscaras, volatile, tiempo de refresco, tablas de segmentos y proceso de compilación/programación. La placa está terminada; el esquema queda para preguntas puntuales de hardware.

18. Esquemático de referencia

Respaldo: no explicar durante el recorrido principal. Usar únicamente ante una pregunta de hardware. El esquema corresponde al diseño del proyecto; confirmar que la selección PCB del firmware coincide con la placa usada. La explicación principal se centra en el software.

19. Dónde encontrar cada responsabilidad

Respaldo para preguntas de código. Abrir firmware/entrenador/entrenador.ino en el editor y localizar la función solicitada. Las diapositivas muestran fragmentos reales, en algunos casos sin comentarios o con saltos de línea adaptados. No se cambió el firmware para preparar esta exposición. Las limitaciones de lectura del DIP, falta de antirrebote explícito y publicación del búfer por bytes pertenecen al código actual.